

API Design

technologies for building APIs

GraphQL

Rest

RPC

But REST APIs are the most widely used

In 1990, Tim Berners-Lee created the World Wide Web. He invented:

HTTP HTML URI First browser & server



But the web grew very fast

Scalability problems

Then in 1993, Roy Fielding proposed constraints to solve this. These constraints became the foundation of REST.

1. Client-Server

UI → Client

Logic & Data → Server

3. Layered System

Layers (proxy, load balancer...)

5. Stateless

Each request contains all info

2. Uniform Interface

Standard way of communication

4. Cache

Responses can be cached

6. Code on Demand

What does REST mean ?

In 2000, Roy Fielding coined the term REST in his PhD dissertation. It stands for **Representational State Transfer**

Representational

A resource can have multiple formats:

JSON HTML XML



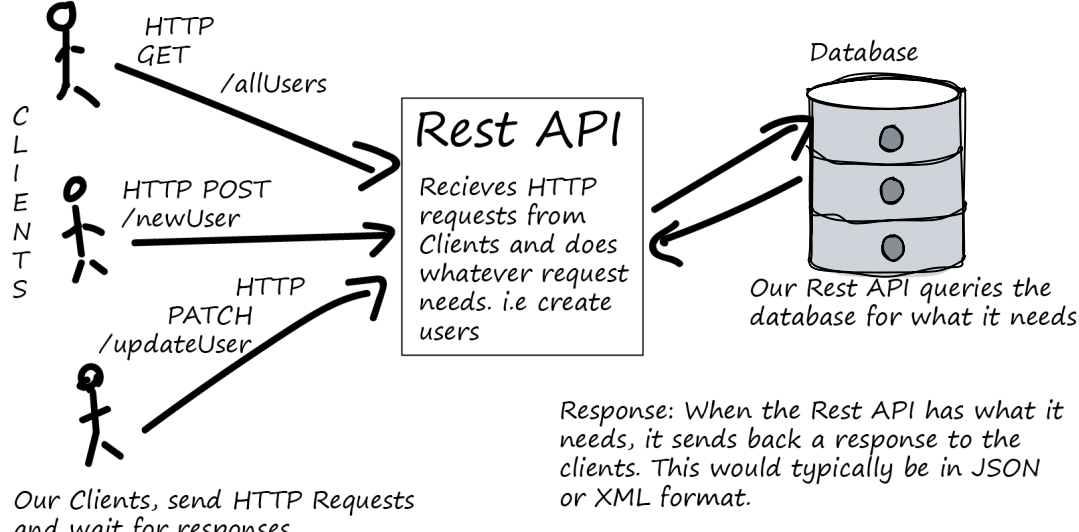
State

Refers to the current condition or attributes of a specific resource at a given moment (the items currently in a shopping cart).

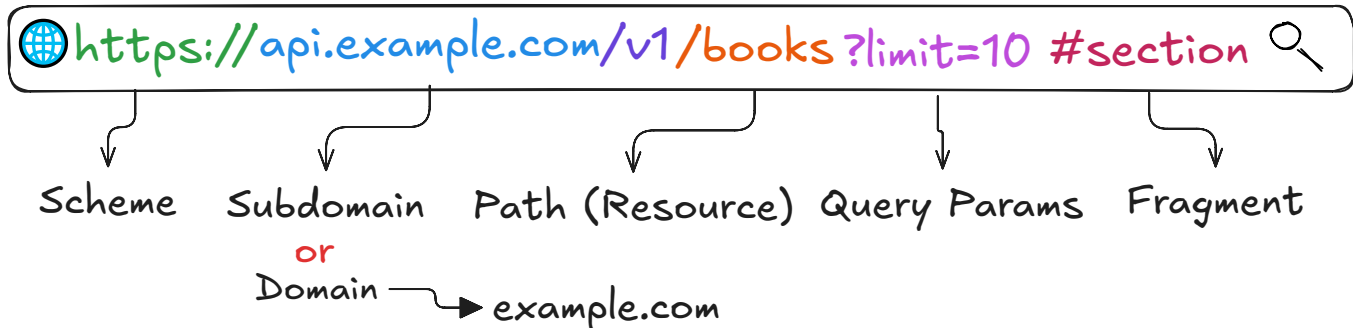
Transfer

The movement of these resource representations and their state between the client and server, utilizing standardized HTTP Methods (GET, POST, PUT, DELETE, etc.).

Rest API Basics



URL Structure & API Routes



The Plural Rule

Always use plural nouns for resources.

/books ✓

/books/123 ✓

/book ✗

Clean URL & Slug Design

harry-potter ✓ Use

Harry Potter ✗ Hyphens (-)

harry_potter ✗ Lowercase

Hierarchical Structure

/books/harry-potter

HTTP Methods (Core of REST)

REST uses HTTP methods to define actions

GET	POST	PUT	PATCH	DELETE
Read only	Create / send data	Update	Update	Remove
تستخدم لجلب البيانات فقط من السيرفر.	تستخدم لإرسال بيانات جديدة للسيرفر.	تستخدم لتحديث البيانات بالكامل.	تستخدم لتحديث جزء من البيانات فقط.	تستخدم لحذف البيانات.
GET /users	POST /users	PUT /users/5	PATCH /users/5	DELETE /users/5
	{ "name": "Mahmoud", "age": 22 }	{ "name": "Mahmoud", "age": 23 }	{ "email": "new@mail.com" }	

Idempotency

Making the same request multiple times

Results in the same outcome

Idempotent

GET

PUT

DELETE

Not Idempotent:

POST

Status Codes

Status Code

1xx / 2xx / 3xx / 4xx / 5xx

The API Design Workflow

Understand product → extract resources → Schema → design API → then code

Step 1: Resources

Resources comes from UI + Business Requirements

Organizations

Projects

Tasks

Step 2: Schema

Organization

{

"id": "number",

"name": "string",

"createdAt": "date"

}

Project

{

"id": "number",

"title": "string",

"organizationId": "number"

}

Task

{

"id": "number",

"title": "string",

"completed": "boolean",

"projectId": "number"

}

Step 3: Relations

1. One-to-One

2. One-to-Many

3. Many-to-Many

Step 4: API Design

(CRUD Operations)

GET /tasks → get all

GET /tasks/1 → get one

POST /tasks → create

PATCH /tasks/1 → update

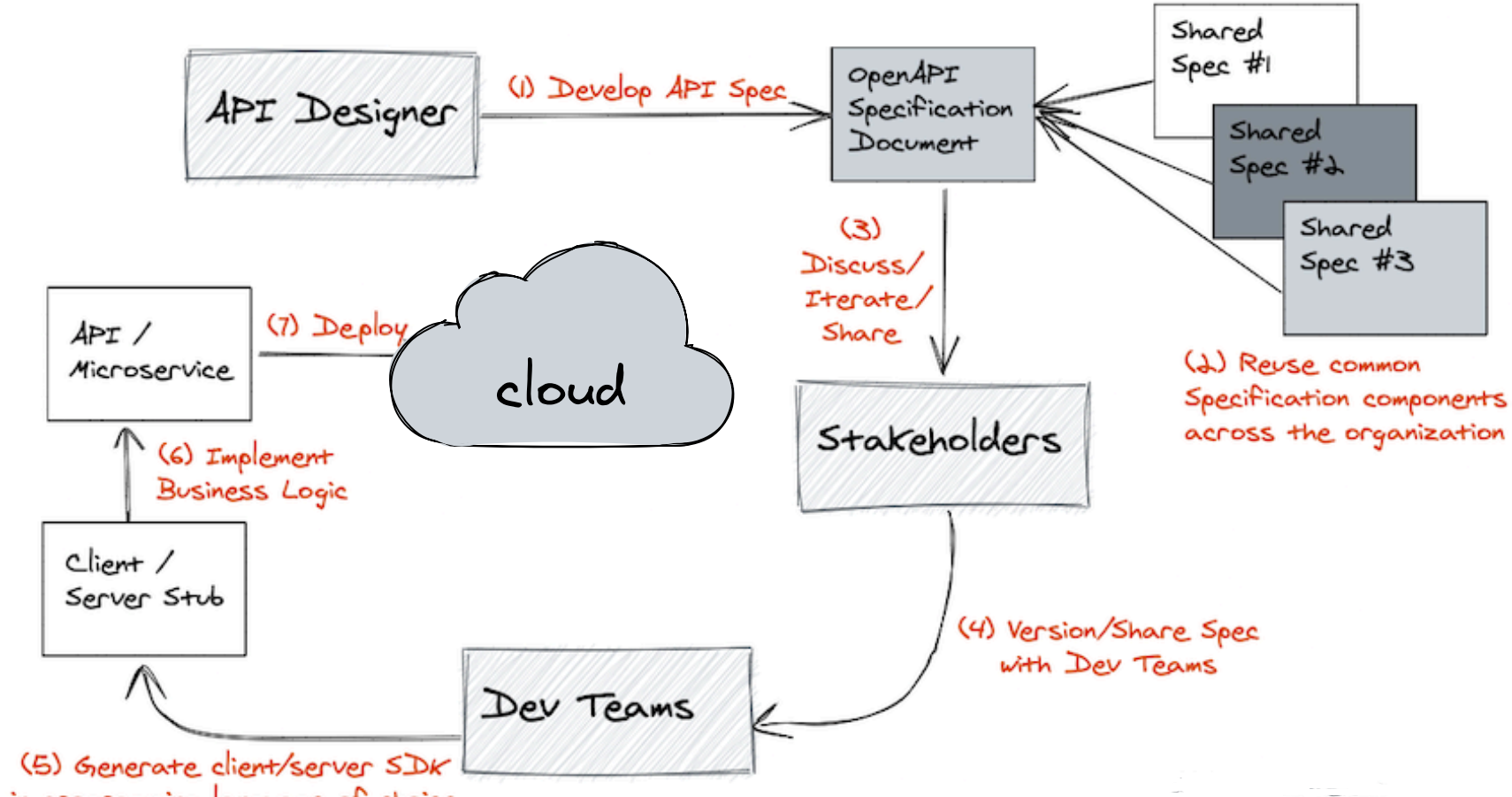
DELETE /tasks/1 → delete

Step 5: API Interface

summary

1. Understand Product (UI)
2. Extract Resources (Nouns)
3. Design Schema (per resource)
4. Define Relations (with real links)
5. Design API (CRUD + endpoints)
6. Then Code

Enterprise level



*** Cross-Links ***

📌 - Related to: JavaScript - API Overview & REST Architecture